DAY 29 ASSIGNMENT - CROSS-SITE SCRIPTING (XSS) FUNDAMENTALS & SAFE LAB

PART A — FUNDAMENTALS

1. What is Cross-Site Scripting (XSS)?

- XSS is a web security vulnerability that allows attackers to inject malicious scripts into web pages.
- These scripts execute in the browsers of users visiting the affected website.

2. Explain reflected XSS.

- Reflected XSS occurs when user input is immediately returned in a web server's response without proper encoding.
- The browser executes the injected script when the malicious request is processed.

3. Explain stored XSS.

- Stored XSS occurs when malicious input is saved on the server, such as in a database or comment section.
- The script executes whenever another user views the affected content.

4. Explain DOM-based XSS at a beginner level.

- DOM-based XSS occurs when JavaScript in a webpage handles user input unsafely and modifies the page.
- The browser executes the injected script because the client-side code processes the input incorrectly.

5. What is reflected input?

- Reflected input is data supplied by a user that appears again in the application's response.
- Example: Search term displayed on search page.

6. Why does reflected input alone not automatically prove XSS?

- Reflected input may be displayed safely as ordinary text.
- XSS occurs only when the input is handled in a way that allows it to execute as code.

7. What is a canary value?

- A canary value is a unique, harmless test string used to identify where user input appears in an application.
- It helps security testers track input through requests and responses.

8. What is output encoding?

- Output encoding converts special characters into safe representations before displaying user input.
- It prevents the browser from interpreting untrusted input as executable code.

9. Why must output encoding match the output context?

- Different contexts, such as HTML, JavaScript and URLs, interpret special characters differently.
- Using the correct encoding ensures that user input remains data rather than executable code.

DAY 29 ASSIGNMENT - CROSS-SITE SCRIPTING (XSS) FUNDAMENTALS & SAFE LAB

10. What is HTML sanitization?

- HTML sanitization removes or modifies potentially dangerous HTML elements and attributes.
- It allows applications to accept HTML while reducing the risk of malicious scripts.

11. What is Content Security Policy?

- CSP is a browser security mechanism that controls which resources a webpage is allowed to load and execute.
- It helps restrict unauthorized scripts and reduce XSS risks.

12. Why should CSP be treated as Defense in Depth rather than the only XSS defense?

- CSP provides an additional layer of protection but may not block every type of XSS attack.
- Applications must still use secure coding practices, proper output encoding and input sanitization where appropriate.

13. How can HttpOnly reduce some XSS impact?

- The HttpOnly cookie attribute prevents JavaScript from accessing cookies through document.cookie.
- This helps protect session cookies from being stolen through certain XSS attacks.

14. Does HttpOnly remove the XSS vulnerability itself? Explain.

- No. HttpOnly only prevents JavaScript from directly accessing protected cookies.

PART B — PORTSWIGGER LAB

Complete: Reflected XSS into HTML context with nothing encoded

Answer:

1. Which application feature was vulnerable: search function

2. Was the input reflected or stored?
The input was reflected immediately in HTTP response.

3. Which XSS type was demonstrated: Reflected Cross site scripting XSS

4. What harmless canary value did you use: JVLXSS29

5. Where did you observe the reflected value: In HTTP response

6. Which Burp Suite feature did you use to resend the request: Burp Suite Repeater.

7. What training proof of concept demonstrated browser-side JavaScript execution?
<script>alert(1)</script>

8. What browser function confirmed execution?

- The JavaScript alert() function displayed a pop-up dialog.

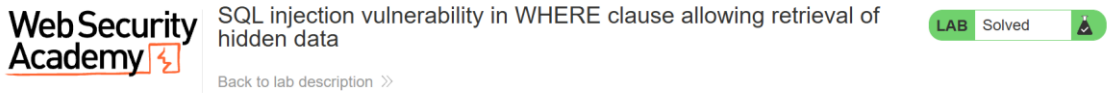
DAY 29 ASSIGNMENT - CROSS-SITE SCRIPTING (XSS) FUNDAMENTALS & SAFE LAB

9. Why was the application vulnerable?

- The application reflected user input directly into the HTML response without proper output encoding, allowing the browser to interpret it as executable JavaScript.

10. Did PortSwigger mark the lab as Solved: YES, solved.

Include one screenshot showing successful lab completion.



PART C — DEFENSIVE ANALYSIS

Fictional application:

Search input is inserted directly into:

<h1>Search results for USER_INPUT</h1>

without output encoding.

Answer:

1. What is the security risk?
The application is vulnerable to Cross-Site Scripting (XSS) because untrusted user input is inserted directly into HTML without encoding, potentially allowing malicious JavaScript to execute in users' browsers.

2. Which XSS category is most likely if the value comes directly from the request?
Reflected XSS, because the input comes from the user's request and is immediately included in the server's response.

3. What primary defensive control should be applied?
Context-aware output encoding should be applied so special characters are treated as data rather than executable HTML or JavaScript.

4. When might HTML sanitization be required?
HTML sanitization is required when an application intentionally allows users to submit HTML content, such as comments or formatted text. It removes unsafe elements and attributes.

5. How could CSP provide additional protection?
Content Security Policy (CSP) can restrict which scripts are allowed to execute on a webpage, providing an additional layer of protection against XSS attacks.

DAY 29 ASSIGNMENT - CROSS-SITE SCRIPTING (XSS) FUNDAMENTALS & SAFE LAB

PART D — COMPARISON

Explain the difference between: SQL Injection and Cross-Site Scripting

Aspect	SQL Injection (SQLi)	Cross-Site Scripting (XSS)
Interpreter involved	SQL database interpreter	Web browser (HTML/JavaScript interpreter)
What untrusted input affects	SQL queries and database operations	Webpage content and browser-side execution
Primary defensive principle	Use parameterized queries (prepared statements)	Use context-aware output encoding

PART E — REFLECTION

Write 5–8 sentences explaining what the PortSwigger lab taught you about the difference between:

The PortSwigger lab taught me that user data should always be treated as untrusted input. User data is ordinary information, such as search terms, that should be displayed as text. Executable browser content includes HTML and JavaScript that the browser can interpret and run. In the lab, I observed that the search input was reflected directly into the HTML response without proper output encoding. This allowed the browser to interpret the injected <script> tag as executable JavaScript instead of ordinary text. I learned that context-aware output encoding helps ensure user input remains data rather than executable content. Overall, the lab taught me the importance of handling user input safely to prevent Cross-Site Scripting (XSS).